On the Interplay of Architecture and Optimization: LSTMs vs. Transformers in Micro-Sequence Generation

Abstract

This study evaluates Long Short-Term Memory (LSTM) networks and causal Transformers on a highly constrained micro-sequence generation task: Chinese name generation. Preliminary baselines using a standard flat learning rate schedule suggested a significant advantage for the LSTM. Targeted optimization reruns, however, suggest that most of this gap was driven by optimizer dynamics rather than a clean architecture-only effect. By applying decoupled weight decay, linear warmup, and cosine annealing to the Transformer, the initial `8.79` perplexity gap collapses to approximately `1.1`. This report details the methodology, the optimization confounders in Transformer training, and a possible residual inductive-bias signal in low-context sequence modeling.

1. Introduction and Task Definition

Modern Natural Language Processing (NLP) is dominated by large-scale, attention-heavy architectures evaluated on long-context tasks [1]. However, not all sequence modeling problems resemble document-scale language generation.

This study focuses on Chinese full name generation, a prototypical **micro-sequence task**. Typical outputs span only two to four characters, yet the generation process remains highly constrained by:

- Surname structure, including compound surnames.
- Local character compatibility and historical stylistic regularities.
- Distributional biases in the source corpus.

This short, constrained geometry tests whether the compact recurrent hidden state of an LSTM [2] may offer a more efficient inductive bias than the self-attention mechanisms of a causal Transformer [1] at a matched parameter scale.

2. The Optimization Confounder

When comparing architectures, identical hyperparameter recipes can inadvertently introduce severe methodological asymmetries. Transformers are notoriously sensitive to early optimizer dynamics. Prior work such as Liu et al. (2019) [3] shows that the adaptive learning rate in Adam can suffer from problematically large variance in the early stages of training. Without a learning rate warmup, this variance can produce unstable early updates - often pushing the model toward sub-optimal regions of the loss landscape, a phenomenon sometimes referred to as the "loss catapult."

Conversely, LSTMs often exhibit higher resilience to naive adaptive gradient updates due to their recurrent state mechanics. To reduce the architecture-versus-optimization confounder, this study evaluates both models under an untuned baseline recipe and targeted tuned recipes incorporating established Transformer training heuristics [4].

3. Methodology and Model Configuration

3.1 Dataset and Tokenization

The source data is derived from the China Biographical Database (CBDB) [5]. The cleaned dataset contains `584,442` full names, strictly partitioned into an `80/10/10` train/validation/test split with split seed `20260419`.

Tokenization uses a naive character-level mapping over individual Unicode characters observed in the training set, augmented with standard control tokens: `<SOS>`, `<EOS>`, `<PAD>`, and `<UNK>`. No subword regularization or byte-pair encoding (BPE) is applied. The resulting shared vocabulary comprises approximately `8.4k` tokens.

3.2 Architectures

Two models were parameter-matched to improve comparability:

1. LSTM Baseline, approximately 5.47M parameters

- Embedding dimension: `160`
- Hidden size: `320`
- Layers: `2`
- Directionality: strictly forward

2. Transformer Baseline, approximately 5.35M parameters

- Embedding dimension: `256`
- Layers: `2`
- Attention heads: `8`
- Feed-forward dimension: `512`
- Positional encoding: learned absolute positional embeddings
- Maximum sequence length: `10`

3.3 Training Recipes

To measure optimization sensitivity, models were trained under the following configurations across three random seeds: `13`, `37`, and `73`.

- **Flat Baseline:** Adam optimizer, flat learning rate `1e-3`, dropout `0.4`.
- **Transformer Tuned:** AdamW optimizer with decoupled weight decay `0.01` [6], `10%` linear warmup followed by cosine decay, dropout `0.1`, and peak learning rates evaluated at `5e-4` and `1e-3`.
- **LSTM Tuned (Symmetry Check):** AdamW optimizer, flat schedule, dropout `0.2`, learning rate `1e-3`. This acts as a methodological control

to check whether the LSTM was similarly bottlenecked by the original hyperparameter selection.

4. Results

All reported metrics evaluate full-data test split performance at sampling temperature `0.8`. Perplexity (PPL) is calculated as `exp(mean cross-entropy)` over non-`PAD` target tokens.

Model and optimization recipe	Mean test PPL	Std dev
LSTM (Flat Adam)	77.9209	0.0342
LSTM Tuned (AdamW, dropout `0.2`)	78.0511	0.0243
Transformer (Flat Adam)	86.7124	0.4884
Transformer Tuned (Peak `5e-4`)	79.0579	0.1839
Transformer Tuned (Peak `1e-3`)	79.0286	0.1572

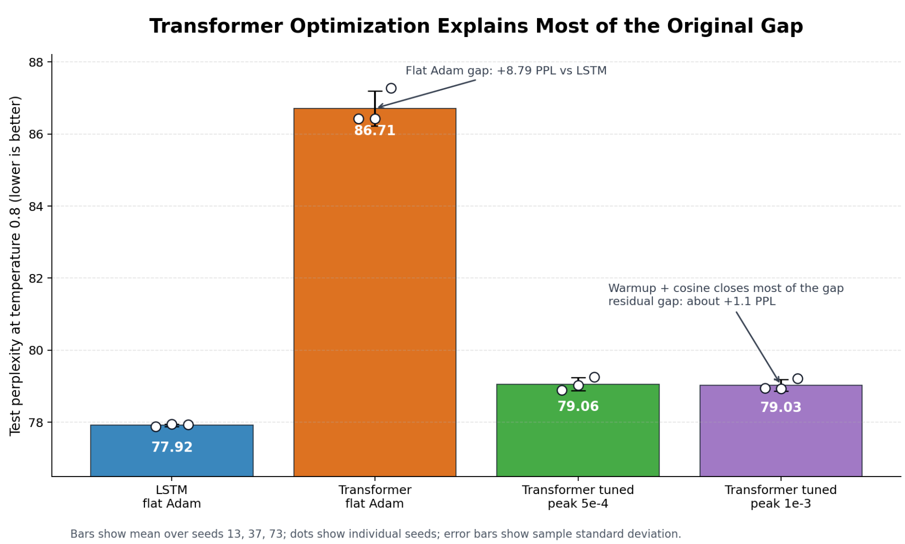

Figure 1: Transformer optimization comparison

Figure 1: Test perplexity comparison showing the optimization sensitivity of the Transformer architecture. The initial gap is mostly mitigated by standard Transformer optimization schedules.

4.1 Generation Quality Metrics

Teacher-forced likelihood fit is correlated with, but distinct from, generation quality. Sample-based metrics, evaluated using pure temperature-scaled categorical sampling, indicate stable decoding behavior across the main reference models:

- **Distinct-2 (bigram diversity):** LSTM `0.8851`; tuned Transformer `0.8655`.
- **Novelty ratio:** LSTM `0.4900`; tuned Transformer `0.4688`.
- **Invalid output ratio:** below `0.001` for both architectures, indicating robust constraint adherence in this evaluation setup.

5. Discussion

The empirical results reframe the initial comparison. The naive flat-Adam baseline yielded a substantial `8.79` perplexity gap favoring the LSTM. However, applying Transformer-appropriate decoupled weight decay and variance-stabilizing warmup allowed the Transformer to recover most of this deficit.

The LSTM symmetry check did not yield reciprocal improvements, moving from `77.92` to `78.05`. This suggests that the initial large gap was driven mainly by the Transformer's optimization constraints rather than a systemic one-sided tuning imbalance.

While Transformer-appropriate optimization closes the majority of the gap, the LSTM retains a small, consistent edge of approximately `1.1` perplexity points across these seeds. Given the highly constrained context window of this specific task, it remains plausible that the recurrent state provides a marginal inductive advantage over self-attention, but this should be treated as a hypothesis rather than a final architecture verdict.

6. Future Work

To assess the residual `1.1` PPL gap more rigorously, future work should establish statistical significance via paired per-sequence log-likelihood testing on the test set. Additionally, exploring weight tying between the input embeddings and the output projection matrix [7] is recommended, as the `8.4k` vocabulary projection currently consumes nearly half of the `5.4M` parameter budget for both models.

Future extensions should also rerun the lower-data `10%` and `30%` ablations with the tuned Transformer recipe, test additional LSTM dropout values, and evaluate whether decoding constraints such as top-k or top-p sampling change the generation-quality tradeoffs.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.

[3] Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., & Han, J. (2019). On the variance of the adaptive learning rate and beyond. *arXiv preprint arXiv:1908.03265.*

[4] Popel, M., & Bojar, O. (2018). Training tips for the transformer model. *The Prague Bulletin of Mathematical Linguistics*, 110(1), 43-70.

[5] China Biographical Database Project (CBDB). Harvard University, Academia Sinica, and Peking University.

[6] Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101.*

[7] Press, O., & Wolf, L. (2017). Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859.*